UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA

Licenciatura em Engenharia de Sistemas Informáticos (LESI-PL)

Unidade Curricular de Integração de Sistemas de Informação

PiiGuard

Auditoria e Anonimização de Dados Pessoais (RGPD) em Sistemas Empresariais Heterogéneos

Trabalho Prático I – Processos ETL

Autora

Brenda Albuquerque (n.º 27209)

Docentes: Luís Ferreira e Óscar Ribeiro

Barcelos, outubro de 2026

Ano letivo 2026/2027

Resumo

Este relatório descreve o PiiGuard, o meu Trabalho Prático I de Integração de Sistemas de Informação. É um conjunto de processos ETL feitos em Pentaho Data Integration que audita e anonimiza dados pessoais de três empresas fictícias, cada uma com os seus sistemas: uma clínica dentária, um ginásio e uma loja de informática.

São três circuitos, e nenhum repete o outro. O primeiro junta dois CSV de clientes com estruturas diferentes, limpa-os com expressões regulares, valida-os e pseudonimiza o NIF. O segundo valida ficheiros SAF-T em XML contra um XSD, extrai as faturas com XPath, cruza-as com a base de dados e exporta-as anonimizadas em JSON, XML, HTML e ZIP. O terceiro recebe mensagens de suporte em tempo real por MQTT, deteta e mascara dados pessoais, alimenta um dashboard Node-RED e audita uma amostra de 10 a 20 % dos registos de cada empresa, avisando-a por email do que encontrou. Uma regra atravessa tudo: não se apaga nada. O que é rejeitado fica em quarentena, com o motivo, e no fim de cada lote conta-se o que entrou e o que saiu.

Palavras-chave: ETL; Pentaho Data Integration; RGPD; anonimização; expressões regulares; MQTT.

Abstract

This report covers PiiGuard, my first practical assignment for Information Systems Integration. PiiGuard is a set of ETL processes built with Pentaho Data Integration that audits and anonymises personal data from three fictitious companies, each with its own systems: a dental clinic, a gym and a computer shop.

There are three pipelines, and none repeats another. The first merges two customer CSV files with different structures, cleans them with regular expressions, validates them and pseudonymises the tax number. The second validates SAF-T XML files against an XSD, extracts the invoices with XPath, matches them against the database and exports them anonymised as JSON, XML, HTML and ZIP. The third receives support messages in real time over MQTT, detects and masks personal data, feeds a Node-RED dashboard and audits a 10 to 20 % sample of each company's records, telling the company by email what was found. One rule runs through all of them: nothing is deleted. Rejected records go to quarantine with the reason, and at the end of each batch the numbers are counted again.

Keywords: ETL; Pentaho Data Integration; GDPR; anonymisation; regular expressions; MQTT.

Índice

Resumo ii
Abstract iii
Índice de figuras v
Índice de tabelas vi
Lista de siglas vii
1. Enquadramento 1
1.1 Equipa, unidade curricular e curso 1
1.2 Objetivos do documento 1
1.3 Estrutura do documento 1
2. Problema 2
3. Estratégia utilizada 3
3.1 Princípios comuns aos três circuitos 4
3.2 Base de dados 5
3.3 Dados de teste 5
4. Transformações 6
4.1 Peças comuns (comum) 6
4.2 Circuito 1 — Clientes 8
4.3 Circuito 2 — Faturas SAF-T 10
4.4 Circuito 3 — Tickets e auditoria 11
5. Jobs 15
5.1 c1_clientes 15
5.2 c2_faturas 16
5.3 c3_receber_tickets 17
5.4 c3_auditoria 18
5.5 Dashboard e emails 19
6. Vídeo de demonstração 20
7. Conclusão e trabalhos futuros 21
7.1 Resultados obtidos 21
7.2 Conclusão 22
7.3 Trabalhos futuros 22
8. Referências bibliográficas 24
Anexo A – Declaração de uso de Inteligência Artificial 25

Índice de figuras

Figura 1 – Arquitetura geral da solução PiiGuard....3
Figura 2 – Transformação abrir_lote....6
Figura 3 – Transformação fechar_lote....6
Figura 4 – Transformação registar_log....7
Figura 5 – Sub-transformação mascarar_pii....8
Figura 6 – Transformação c1_01_extrair (extração para raw)....8
Figura 7 – Transformação c1_02_validar_carregar (limpeza, validação e carga)....10
Figura 8 – Transformação c2_01_ler_saft (leitura do SAF-T com XPath)....10
Figura 9 – Transformação c2_02_exportar (exportação anonimizada)....11
Figura 10 – Transformação c2_03_resumo_auditoria....11
Figura 11 – Transformação c3_01_receber_mqtt (consumidor MQTT)....12
Figura 12 – Transformação c3_01b_processar_ticket....12
Figura 13 – Transformação c3_02_auditar_amostra (amostragem)....13
Figura 14 – Transformação c3_03_notificar (email às empresas)....14
Figura 15 – Job c1_clientes....15
Figura 16 – Job c2_faturas....17
Figura 17 – Job c3_receber_tickets....18
Figura 18 – Job c3_auditoria....18
Figura 19 – Dashboard Node-RED com os alertas do C3....19
Figura 20 – QR Code do vídeo de demonstração....20

Índice de tabelas

Tabela 1 – Enquadramento do trabalho....1
Tabela 2 – Empresas do cenário e respetivas fontes de dados....2
Tabela 3 – Comparação dos três circuitos....3
Tabela 4 – Schemas da base de dados piiguard (PostgreSQL)....5
Tabela 5 – Transformações comuns e reutilizáveis....6
Tabela 6 – Expressões regulares usadas para detetar dados pessoais....7
Tabela 7 – Jobs e respetivos parâmetros....15
Tabela 8 – Resultados obtidos e conferência com o mapa de estragos....21
Tabela 9 – Testes de robustez....22
Tabela 10 – Utilização de Inteligência Artificial no trabalho....25

Lista de siglas

CC	Cartão de Cidadão
CRM	*Customer Relationship Management*
CSV	*Comma-Separated Values*
ETL	*Extract, Transform, Load*
IBAN	*International Bank Account Number*
ISI	Integração de Sistemas de Informação
JSON	*JavaScript Object Notation*
LESI-PL	Licenciatura em Engenharia de Sistemas Informáticos (pós-laboral)
MQTT	*Message Queuing Telemetry Transport*
NIF	Número de Identificação Fiscal
PDI	Pentaho Data Integration
PII	*Personally Identifiable Information (dados pessoais)*
RGPD	Regulamento Geral sobre a Proteção de Dados
SAF-T	*Standard Audit File for Tax Purposes*
SHA-256	*Secure Hash Algorithm (256 bits)*
SMTP	*Simple Mail Transfer Protocol*
SQL	*Structured Query Language*
UC	Unidade curricular
UPCA	Universidade Politécnica do Cávado e do Ave
XML	*Extensible Markup Language*
XPath	*XML Path Language*
XSD	*XML Schema Definition*
XSLT	*Extensible Stylesheet Language Transformations*

1. Enquadramento

1.1 Equipa, unidade curricular e curso

Este é o relatório do Trabalho Prático I de Integração de Sistemas de Informação (ISI), da Licenciatura em Engenharia de Sistemas Informáticos (LESI-PL), na Escola Superior de Tecnologia da Universidade Politécnica do Cávado e do Ave (UPCA). A Tabela 1 reúne os dados do enquadramento.

Tabela 1 – Enquadramento do trabalho

Item	Descrição
Autora	Brenda Santos de Albuquerque (n.º 27209), trabalho individual
Unidade curricular	Integração de Sistemas de Informação (ISI)
Curso	Licenciatura em Engenharia de Sistemas Informáticos (LESI-PL), Escola Superior de Tecnologia, UPCA
Ano letivo	2026/2027
Docentes	Luís Ferreira e Óscar Ribeiro
Plataformas	Pentaho Data Integration 9.4 (Spoon e Kitchen), PostgreSQL, Mosquitto (MQTT), Node-RED com Dashboard, Gmail (SMTP), GitHub
Repositório	https://github.com/brendaalbuq/piiguard-gdpr-etl

Fonte: Elaboração própria.

1.2 Objetivos do documento

O trabalho aplica processos ETL (*Extract*, *Transform*, *Load*) à integração de sistemas ao nível dos dados. O cenário escolhido é uma **auditoria a dados**, o primeiro dos exemplos que o enunciado dá para processos deste tipo. O relatório conta o problema, a estratégia, as transformações e os Jobs construídos no Pentaho, os resultados e o vídeo de demonstração.

1.3 Estrutura do documento

O capítulo 2 apresenta o problema e o cenário. O capítulo 3 trata da estratégia, da arquitetura, dos princípios comuns e da base de dados. Nos capítulos 4 e 5 estão as transformações e os Jobs, com os diagramas. O capítulo 6 indica o vídeo, e o capítulo 7 reúne resultados, conclusão e trabalhos futuros. Depois vêm as referências bibliográficas e, em anexo, a declaração de uso de Inteligência Artificial.

2. Problema

As empresas guardam dados pessoais em sistemas diferentes, em formatos diferentes e, com frequência, em sítios onde não deviam estar. Muitas nem sabem onde estão esses dados, nem em que estado. O Regulamento Geral sobre a Proteção de Dados (RGPD) exige saber que dados pessoais existem, reduzi-los ao necessário e protegê-los, por exemplo com pseudonimização (Regulamento (UE) 2016/679, art. 32.º). Os dados de saúde merecem ainda mais cuidado (art. 9.º).

O cenário é **fictício**: um auditor de RGPD recebe dados de três empresas clientes, descritas na Tabela 2.

Tabela 2 – Empresas do cenário e respetivas fontes de dados

Empresa	Sistema de origem	Formato	Problemas típicos
A – Clínica Dentária Sorriso do Minho	CRM de clientes, faturação (SAF-T) e suporte	CSV com `;` e colunas em português, XML SAF-T, JSON	Dados de saúde nas observações, NIF em vários formatos
B – Ginásio Movimento Braga	CRM de clientes e suporte	CSV com `,` e colunas em inglês, JSON	Estrutura diferente, emails inválidos, duplicados
C – TechCávado Informática	Faturação (SAF-T) e suporte	XML SAF-T, JSON	XML que não respeita o XSD, clientes que não existem no CRM

Fonte: Elaboração própria.

O trabalho pretende demonstrar que é possível:

1. integrar fontes heterogéneas: ficheiros CSV e XML, uma base de dados PostgreSQL e mensagens MQTT recebidas em tempo real;
2. limpar e normalizar com **expressões regulares** dados estragados de propósito (NIF com espaços, pontos ou prefixo `PT`; telefones com `+351`, `00351` ou `(+351)`; datas em quatro formatos; códigos postais sem hífen; nomes com acentos corrompidos);
3. **nunca destruir dados**: o original fica guardado, o que é rejeitado fica em quarentena com o motivo, e a contagem do lote prova para onde foi cada registo;
4. detetar e mascarar dados pessoais escondidos em texto livre, auditar uma amostra de **10 a 20 %** dos registos de cada empresa e **informar cada empresa por email** do que foi encontrado, sem enviar dados pessoais;
5. mostrar os resultados para além de um CSV: dashboard em tempo real, relatório HTML, ficheiros exportados e email.

3. Estratégia utilizada

A solução, a que dei o nome **PiiGuard**, tem **três circuitos diferentes**. Cada um é um Job do Pentaho Data Integration (Casters et al., 2010) que orquestra transformações pequenas e tem fonte, problema, operadores e destino próprios. Não são três cópias do mesmo fluxo. Três foi uma escolha deliberada: as aulas pediram pelo menos três circuitos diferentes e sem complexidade a mais, e assim consigo explicar cada um por inteiro. A Figura 1 mostra a arquitetura geral e a Tabela 3 compara os circuitos.

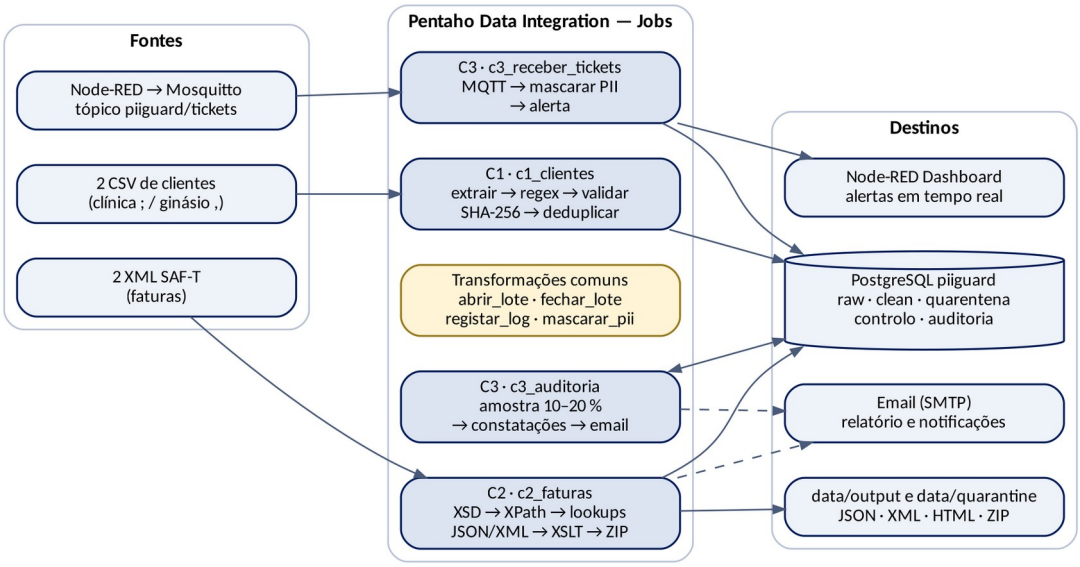


Figura 1 – Arquitetura geral da solução PiiGuard

Fonte: Elaboração própria.

Tabela 3 – Comparação dos três circuitos

	C1 — Clientes	**C2 — Faturas SAF-T**	**C3 — Tickets de suporte**
Fonte e formato	2 CSV com separadores e colunas diferentes	2 XML SAF-T + PostgreSQL (CRM)	JSON via MQTT (Mosquitto), publicado pelo Node-RED
Modo	Síncrono, por lote	Síncrono, por lote	Assíncrono, por evento, mais auditoria a pedido
Pergunta a que responde	Os dados que chegaram são utilizáveis?	As faturas expõem dados pessoais? Batem certo com o CRM?	Há dados pessoais escondidos em texto livre?
Operadores principais	CSV input, Add constants, Select values (schema mapping), Concat fields, JavaScript, String operations, 9 × Replace in string (regex), Data Validator, Add a checksum (SHA-256), Mapping, Sort rows, Unique rows, Table output	XSD Validator, Get data from XML (XPath), Stream lookup, Database lookup, Filter rows, JSON output, XML output, Sort rows, Group by, XSLT, Zip, Mail	MQTT Consumer, Get records from stream, JSON input, Database lookup, Filter rows, Mapping, MQTT Producer, Generate random value, Add value fields changing sequence, Memory group by, Mail (por linha), Execute SQL

	C1 — Clientes	C2 — Faturas SAF-T	C3 — Tickets de suporte
Destino	`raw` → `clean` + `quarentena`	JSON/XML anonimizados, relatório HTML, ZIP, email	Base de auditoria, dashboard em tempo real, email a cada empresa

Fonte: Elaboração própria.

3.1 Princípios comuns aos três circuitos

- **Não destruir dados.** O que chega fica gravado tal como veio, em `raw`, tudo em texto, para a entrada nunca rejeitar nada. As tabelas `raw` e `quarentena` aceitam só inserções: um *trigger* recusa `UPDATE` e `DELETE`. Onde um exemplo das aulas usaria um passo *Dummy*, aqui os registos seguem para a quarentena, com o motivo. E um XML que falha o XSD é copiado para `data/quarantine`, nunca apagado.
- **Dizer para onde foram os dados.** Cada execução abre um lote (`controlo.lote`). No fim, a reconciliação confirma que `entrada = válidos + quarentena + duplicados`. Se a conta não fechar, o Job aborta.
- **Transações.** A transformação que carrega `clean` e `quarentena` é transacional, com uma só ligação à base de dados: ou entra tudo, ou nada (*rollback*). O original continua em `raw` e pode ser reprocessado.
- **Temporalidade.** Lotes, log de execução (`controlo.etl_log`), histórico de auditorias, amostras, constatações e notificações têm todos data.
- **Reutilização.** As operações comuns vivem em `transformations/comum` e vários Jobs chamam-nas: `abrir_lote`, `fechar_lote`, `registar_log` e a sub-transformação `mascarar_pii`, que o C1 usa nas observações e o C3 nas mensagens.
- **Proteção de dados.** O NIF é pseudonimizado com SHA-256 antes de qualquer exportação. O texto livre é mascarado (`[NIF]`, `[IBAN]`, `[DADO_SAUDE]`…), e tanto os emails como o dashboard só mostram texto mascarado. As passwords ficam fora dos ficheiros, em variáveis do `kettle.properties`.
- **Envio opcional.** Os Jobs com email têm o parâmetro `ENVIAR_EMAIL` (S/N), como no exemplo *FazTudo* das aulas. Com `N`, as notificações ficam `PENDENTE` e seguem na execução seguinte.

3.2 Base de dados

Tabela 4 – Schemas da base de dados piiguard (PostgreSQL)

Schema	Conteúdo	Regra
`ref`	Empresas e motivos de rejeição	Tabelas de referência
`controlo`	Lotes, reconciliação (`v_reconciliacao`) e log	Uma linha por execução
`raw`	Clientes e mensagens MQTT tal como chegaram	Só inserção (*trigger*)
`clean`	Clientes normalizados (com histórico de versão) e faturas pseudonimizadas	Dados utilizáveis
`quarentena`	Registos rejeitados, com campo, motivo e registo original	Só inserção (*trigger*)
`auditoria`	Tickets mascarados, execuções, candidatos da amostra (selecionados ou não), constatações e notificações	Resultado da auditoria

Fonte: Elaboração própria.

Os scripts SQL estão em `src/sql`. O `01_criar_estrutura.sql` cria a base de raiz e recusa correr se os schemas já existirem, para nunca apagar nada. Numa base que já existe usa-se o `03_migrar_sem_perda.sql`, que só acrescenta colunas, tabelas e restrições. O `04_verificar_resultados.sql` reúne as consultas de verificação e não altera nada.

3.3 Dados de teste

As três empresas e os dados são fictícios e saem de um script Python com semente fixa (`src/scripts/gerar_dados.py`), por isso o resultado é sempre o mesmo. Os erros foram injetados de propósito, e cada um está em `data/expected/mapa_estragos.csv`, ao lado do que o ETL deve fazer com ele. São 205 estragos, entre clientes, faturas e tickets. Foi este mapa que serviu para conferir os resultados (secção 7.1).

4. Transformações

As transformações estão em `dataintegration/transformations`, numa pasta por circuito mais uma pasta `comum` com as peças partilhadas. Cada uma faz uma única coisa; quem as encadeia são os Jobs (capítulo 5). A ligação à base de dados (`piiguard_db`) usa só variáveis (`${DB_HOST}`, `${DB_PASS}`…), definidas no `kettle.properties`.

4.1 Peças comuns (`comum`)

Tabela 5 – Transformações comuns e reutilizáveis

Transformação	O que faz	Operadores
`abrir_lote`	Cria uma linha em `controlo.lote` e devolve o número do lote, que fica na variável `LOTE_ID` para o resto do Job	Table input (`INSERT … RETURNING lote_id`), Set variables
`fechar_lote`	Conta quantos registos entraram e para onde foram (válidos, quarentena, duplicados) e marca o lote como `SUCESSO` ou `FALHOU`	Execute SQL script
`registar_log`	Escreve uma linha em `controlo.etl_log` (circuito, passo, nível, mensagem). É chamada no fim de cada Job, no caminho de sucesso e no de erro; com nível `ERRO`, marca também o lote como `FALHOU`	Execute SQL script
`mascarar_pii`	Sub-transformação (Mapping) que recebe um campo de texto livre, troca os dados pessoais por etiquetas e classifica a gravidade	Mapping input/output, 8 × Replace in string, JavaScript

Fonte: Elaboração própria.

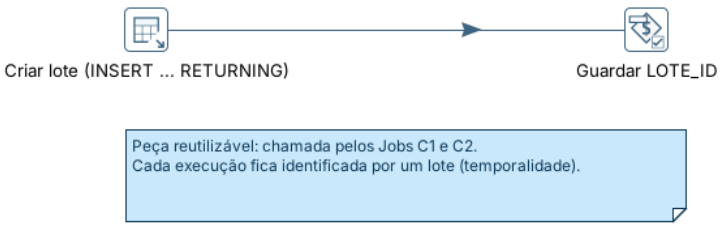


Figura 2 – Transformação abrir_lote

Fonte: Elaboração própria (captura do Pentaho Spoon).

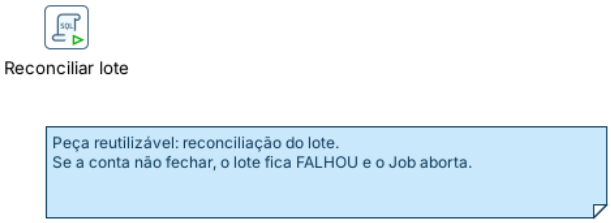


Figura 3 – Transformação fechar_lote

Fonte: Elaboração própria (captura do Pentaho Spoon).

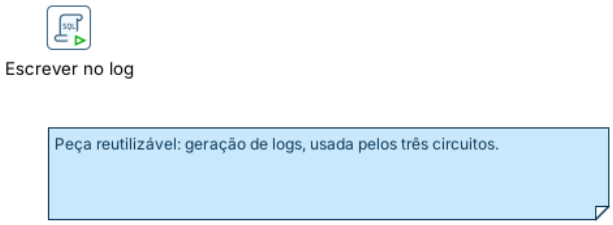


Figura 4 – Transformação registar_log
Fonte: Elaboração própria (captura do Pentaho Spoon).

A `mascarar_pii` é a peça mais reaproveitada: o C1 usa-a nas observações dos clientes e o C3 nas mensagens dos tickets. Cada tipo de dado tem a sua regex, num passo à parte:

Tabela 6 – Expressões regulares usadas para detetar dados pessoais

Etiqueta	Regex	Exemplo apanhado
`[EMAIL]`	`[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)+`	joana.m@gmail.com
`[IBAN]`	`PT\d{2}(\s?\d){21}`	PT50 0035 0123 …
`[CC]`	`\b\d{8}\s?\d\s?[A-Z]{2}\d\b`	12345678 9 ZZ1
`[TELEFONE]`	`(\(?(\+|00)351\)?[\s-]?)?\b9[1236](\s?-?\d){7}\b`	+351 912 345 678
`[NIF]`	`\b[1235689]\d{2}[\s.]?\d{3}[\s.]?\d{3}\b`	245.123.456
`[CODIGO_POSTAL]`	`\b\d{4}-\d{3}\b`	4750-123
`[MORADA]`	`\b(Rua|Avenida|Av\.|Largo|Travessa|Praça)\s[^,]+,\s*\d+`	Rua do Souto, 12
`[DADO_SAUDE]`	palavras-chave: diabetes, alergia, alérgico/a, penicilina, anticoagulantes, ficha clínica, asma, hipertensão, grávida	"alergia à penicilina"

Fonte: Elaboração própria.

No fim, um passo JavaScript junta a lista de tipos encontrados (`tipos_pii`), diz se o texto tinha dados pessoais (`tem_pii`) e atribui a gravidade: **ALTA** com dados de saúde (artigo 9.º do RGPD), IBAN ou CC; **MEDIA** com NIF ou morada; **BAIXA** nos restantes casos.

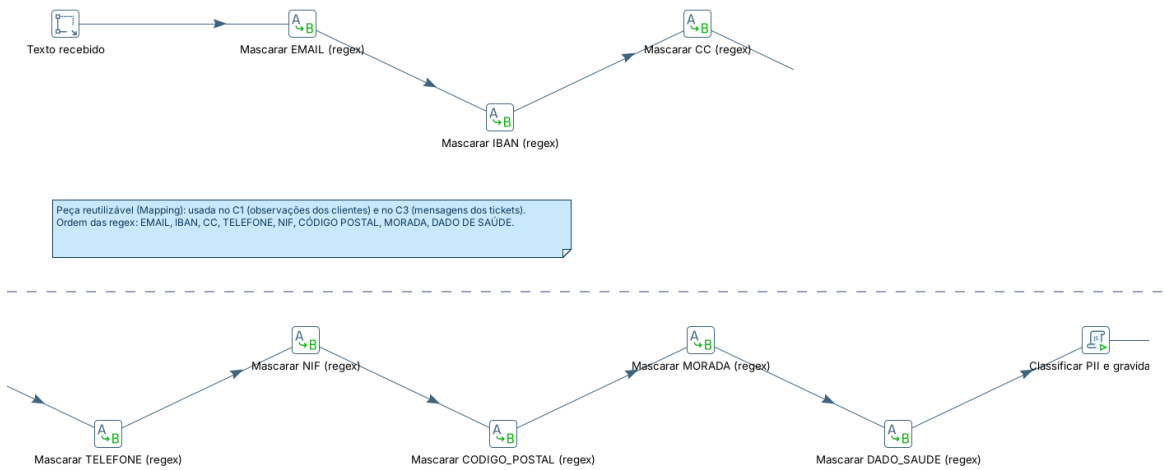


Figura 5 – Sub-transformação mascarar_pii
Fonte: Elaboração própria (captura do Pentaho Spoon).

4.2 Circuito 1 — Clientes

`c1_01_extrair` — extração para a zona `raw`. Os dois CSV diferem nos separadores e nos nomes das colunas: a clínica usa `;` e colunas em português, o ginásio usa `,` e colunas em inglês. Cada ficheiro tem o seu *CSV file input*; um *Add constants* acrescenta a empresa e o nome do ficheiro; um *Select values* faz o **schema mapping** para uma estrutura comum; o número do lote vem de *Get variables*. Tudo é gravado em `raw.cliente` como texto, sem validação nenhuma, para nada se perder à entrada.

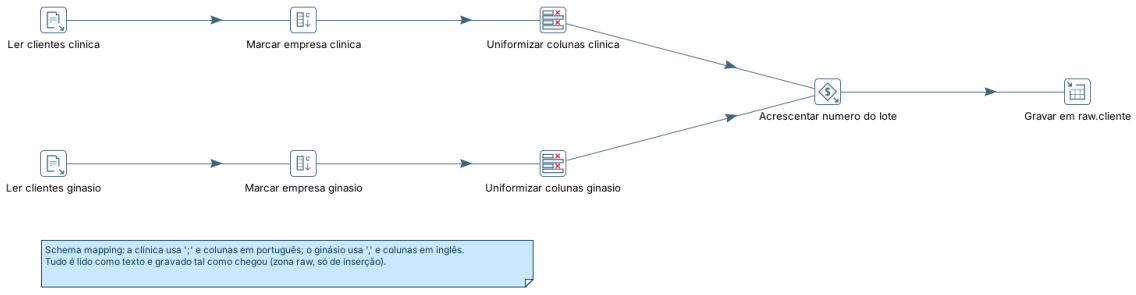


Figura 6 – Transformação c1_01_extrair (extração para raw)
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c1_02_validar_carregar` — limpeza, validação e carga (transacional). É a transformação principal do C1. Lê o lote de `raw` e:

1. guarda o registo original numa só coluna (*Concat fields*), para o acompanhar até à quarentena, se for esse o destino;

2. corrige acentos corrompidos (JavaScript) e apara espaços e maiúsculas (*String operations*);
3. normaliza com **uma regex por passo** (*Replace in string*): o NIF (sem espaços, pontos nem `PT`), o telefone (sem separadores nem indicativo, voltando a pôr `+351`), o código postal, as datas `dd/mm/aaaa`, `d-m-aa` e `dd.mm.aaaa` (passam a `aaaa-mm-dd`), a hora da data de registo, os espaços repetidos no nome e a separação entre nome e apelidos;
4. confirma que a data existe mesmo (JavaScript com `LocalDate`): 31 de fevereiro passa a `INVALIDA`;
5. valida com o *Data Validator*: NIF obrigatório com 9 dígitos, email, telemóvel `^\+3519\d{8}$`, código postal `^\d{4}-\d{3}$` e datas; o *error handling* leva os registos que falham para a quarentena, com o campo e o motivo;
6. pseudonimiza o NIF com **SHA-256** (*Add a checksum*) e mascara as observações com a sub-transformação `mascarar_pii`;
7. ordena por empresa e NIF e remove duplicados (*Unique rows*): os duplicados também vão para a quarentena, com o motivo `DUPLICADO`; é aqui que aparecem os duplicados "disfarçados", que só ficam iguais depois de o NIF ser normalizado;
8. converte as datas e grava em `clean.cliente`; os rejeitados vão para `quarentena.registo` com o registo original.

A transformação usa uma só ligação à base de dados (*Make the transformation database transactional*). Se um passo falhar, nada entra em `clean` nem em `quarentena`, e o lote pode ser reprocessado a partir de `raw`.

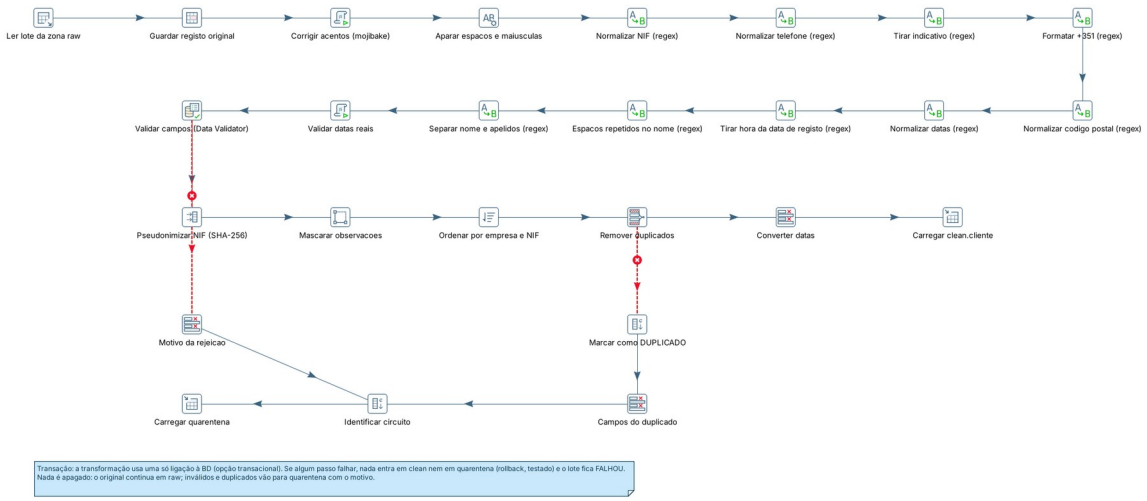


Figura 7 – Transformação c1_02_validar_carregar (limpeza, validação e carga)
Fonte: Elaboração própria (captura do Pentaho Spoon).

4.3 Circuito 2 — Faturas SAF-T

`c2_01_ler_saft` — leitura do XML com XPath. Recebe o ficheiro por parâmetro e é chamada uma vez por cada ficheiro válido. Um *Get data from XML* percorre `/AuditFile/SourceDocuments/SalesInvoices/Invoice` e outro percorre os clientes; o NIF da empresa vem de `../../../Header/CompanyID`. Um *Stream lookup* junta cada fatura ao seu cliente, o NIF passa a SHA-256 e dois *Database lookup* buscam o cliente ao CRM (`clean.cliente`, pelo hash do NIF) e a empresa a `ref.empresa`. As faturas são gravadas em `clean.fatura`; as que não têm cliente no CRM geram uma constatação `CLIENTE_SEM_CRM` (*Filter rows*).

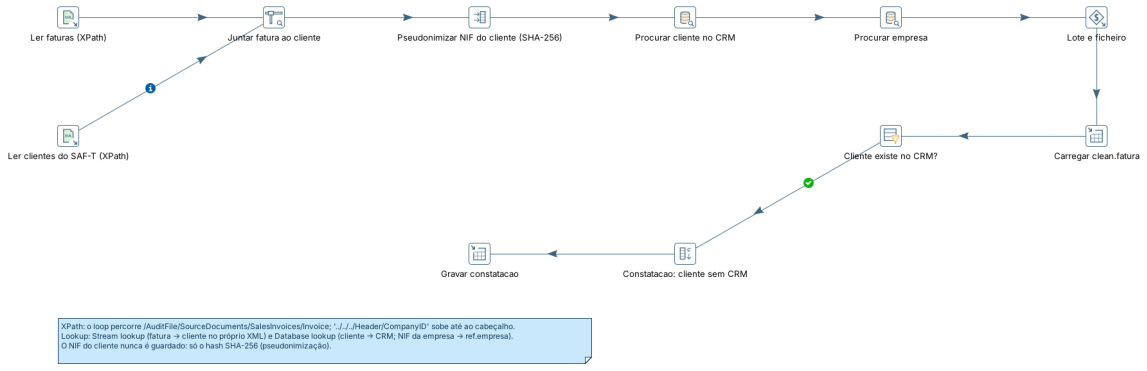


Figura 8 – Transformação c2_01_ler_saft (leitura do SAF-T com XPath)
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c2_02_exportar` — exportação anonimizada. Lê as faturas do lote e escreve, em paralelo, um JSON e um XML sem dados pessoais (só o hash do NIF), mais um JSON com os totais

por mês e por empresa (*Sort rows + Group by*). O número do lote entra no nome dos ficheiros.

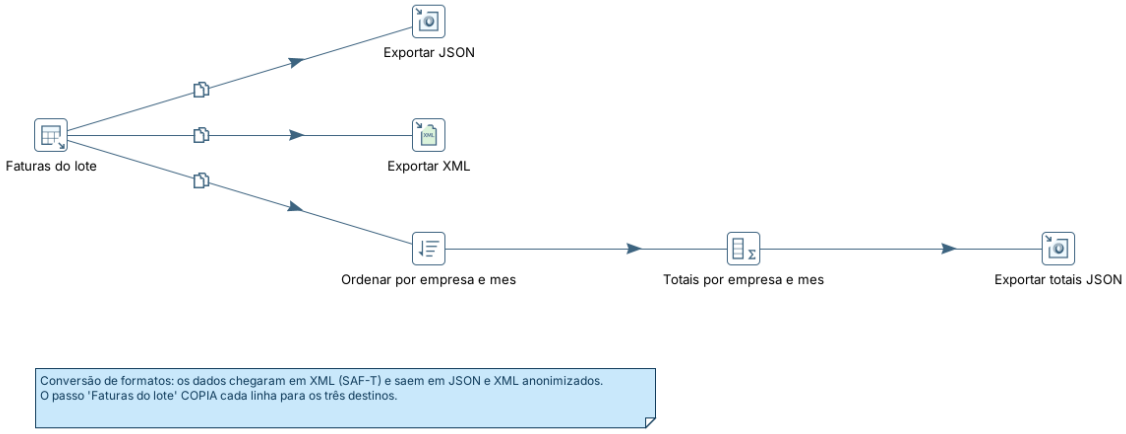


Figura 9 – Transformação c2_02_exportar (exportação anonimizada)
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c2_03_resumo_auditoria` — resumo para o relatório. Uma consulta SQL junta quatro secções (lotes e reconciliação, quarentena por motivo, constatações por empresa e faturas exportadas) e grava-as num XML. Depois, o Job transforma esse XML em HTML com XSLT.

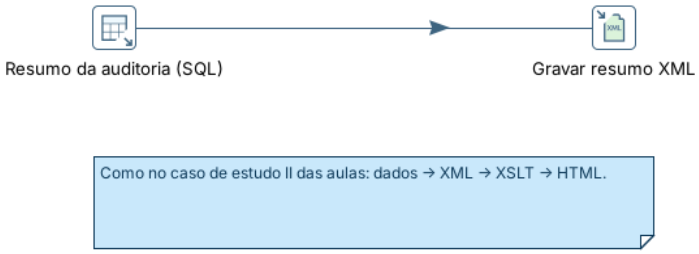


Figura 10 – Transformação c2_03_resumo_auditoria
Fonte: Elaboração própria (captura do Pentaho Spoon).

4.4 Circuito 3 — Tickets e auditoria

`c3_01_receber_mqtt` — consumidor MQTT. O *MQTT Consumer* subscreve o tópico `piiguard/tickets` no Mosquitto e, de 2 em 2 segundos, entrega as mensagens recebidas à transformação filha. Fica a correr até alguém o parar, e é aí que está a parte assíncrona do trabalho.

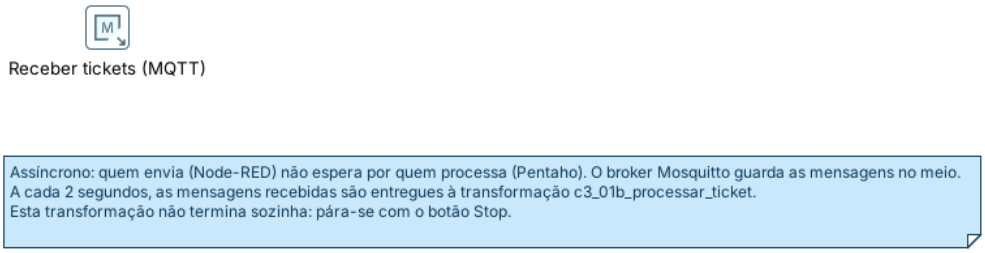


Figura 11 – Transformação c3_01_receber_mqtt (consumidor MQTT)
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c3_01b_processar_ticket` — processamento de cada mensagem. Grava a mensagem original em `raw.ticket` e lê os campos do JSON (*JSON input*). Manda para a quarentena as mensagens sem texto (`CAMPO_OBRIGATORIO_NULO`) e as repetidas (`EVENTO_REPETIDO`, com um *Database lookup* ao identificador do evento). O texto é mascarado com `mascarar_pii` e guardado em `auditoria.ticket`. Se tinha dados pessoais, um passo JavaScript monta um alerta (empresa, gravidade, tipos encontrados e texto já mascarado) e o *MQTT Producer* publica-o em `piiguard/alertas`, que alimenta o dashboard.

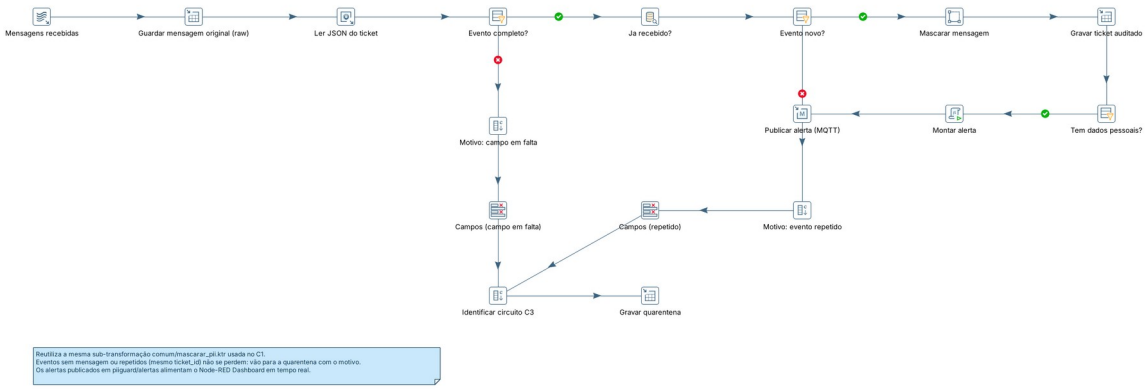


Figura 12 – Transformação c3_01b_processar_ticket
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c3_02_auditar_amostra` — amostragem de 10–20 %. Junta os registos auditáveis dos três circuitos de cada empresa (clientes válidos do C1, faturas do C2 e tickets do C3) e dá um número aleatório a cada um (*Generate random value*). Ordena por empresa e por esse número e numera os registos dentro de cada empresa (*Add value fields changing sequence*). Em paralelo, conta os registos por empresa (*Memory group by*) e calcula o limite `ceil(total × PCT / 100)`; ficam selecionados os registos com número até esse limite. Todos os candidatos são gravados em `auditoria.amostra`, com `selecionado` verdadeiro ou falso, para a seleção poder ser conferida mais tarde. Só os selecionados são analisados: os que têm dados pessoais em texto livre geram as constatações

`DADO_SAUDE_EM_TEXTO_LIVRE` ou `PII_EM_TEXTO_LIVRE`, e as faturas de clientes que não existem no CRM geram `CLIENTE_SEM_CRM`.

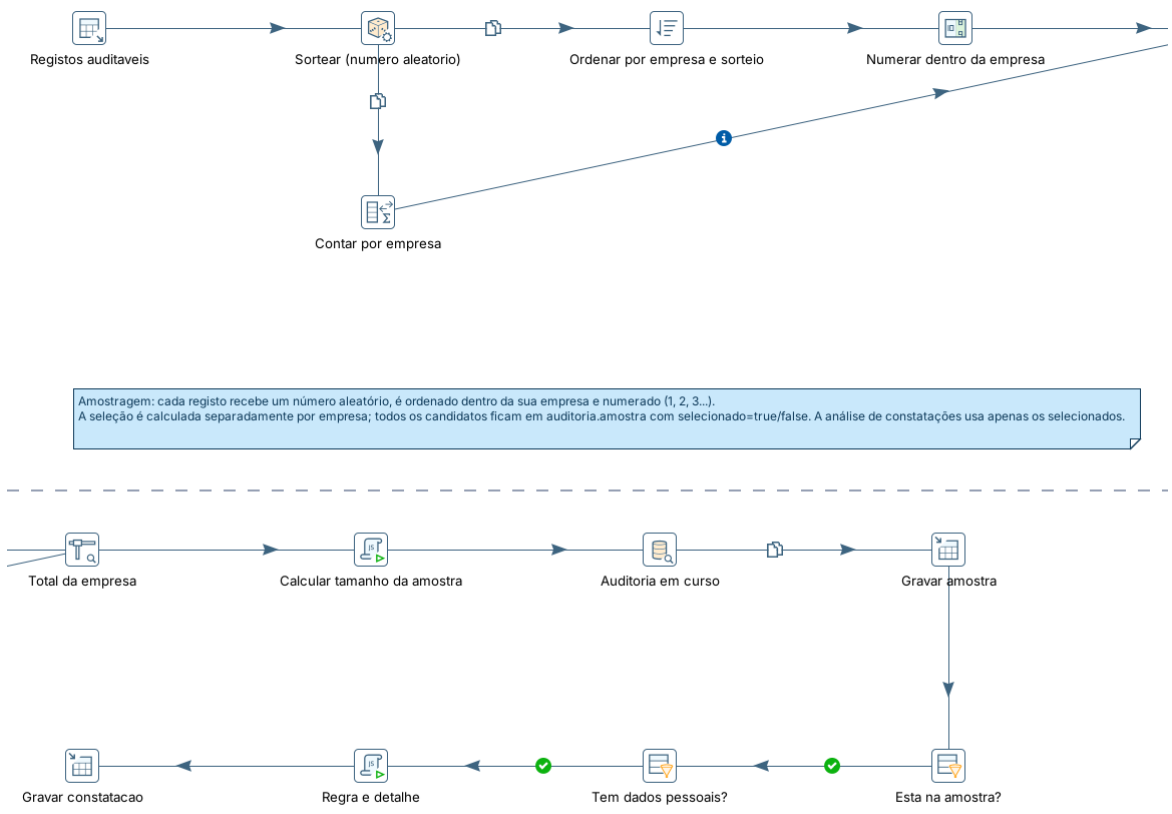


Figura 13 – Transformação c3_02_auditar_amostra (amostragem)
Fonte: Elaboração própria (captura do Pentaho Spoon).

`c3_03_notificar` — email a cada empresa. Lê as notificações pendentes ou falhadas (com menos de 3 tentativas) e, para cada empresa, monta o assunto, que leva o número da auditoria para se perceber um eventual reenvio, e um corpo em HTML com a percentagem pedida, a observada e o número de constatações por tipo e gravidade, **sem nenhum dado pessoal**. O passo *Mail* envia por SMTP com SSL, com as definições do servidor vindas de variáveis (*Get variables*) e não do ficheiro. Se o envio correr bem, a notificação passa a `ENVIADO`; se falhar, o *error handling* marca-a como `FALHOU` e soma uma tentativa.

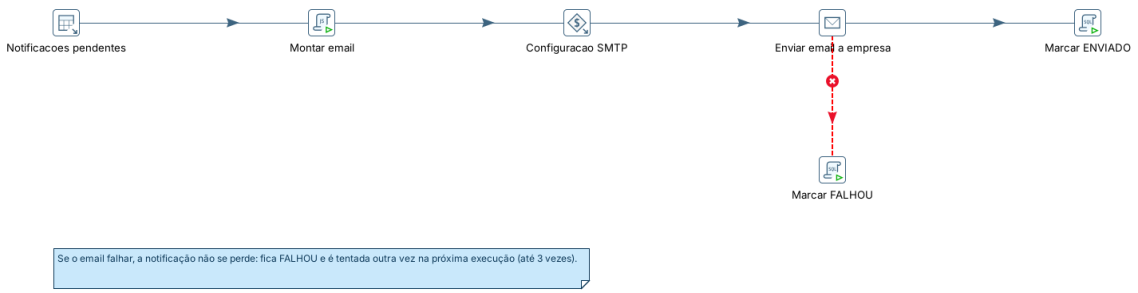


Figura 14 – Transformação c3_03_notificar (email às empresas)
Fonte: Elaboração própria (captura do Pentaho Spoon).

5. Jobs

Há um Job por circuito, com dois no C3: um recebe os tickets, o outro faz a auditoria. Os Jobs estão em `dataintegration/jobs` e chamam as transformações por caminho relativo (`${Internal.Entry.Current.Directory}/../transformations/…`), por isso a pasta do projeto pode estar em qualquer lado. Nos Jobs, a seta verde é o caminho de sucesso e a vermelha o de falha. Todas as falhas passam por `registar_log` (nível `ERRO`) antes do *Abort*, para ficar escrito onde o circuito parou.

Tabela 7 – Jobs e respetivos parâmetros

Job	Parâmetros	Execução
`c1_clientes`	—	Síncrona, por lote
`c2_faturas`	`ENVIAR_EMAIL` (S/N, por omissão N)	Síncrona, por lote
`c3_receber_tickets`	—	Assíncrona, fica a correr
`c3_auditoria`	`PCT` (10–20, por omissão 15), `ENVIAR_EMAIL` (S/N)	Síncrona, a pedido ou agendada

Fonte: Elaboração própria.

5.1 `c1_clientes`

`Start` → **Abrir lote** (`abrir_lote`, circuito C1) → **Extrair CSV para raw** (`c1_01`) → **Validar, limpar e carregar** (`c1_02`) → **Fechar lote** (`fechar_lote`) → **A conta fecha?** → **Log sucesso** → `Success`.

A entrada *A conta fecha?* (*Evaluate rows number in a table*) consulta a vista `controlo.v_reconciliacao` e só deixa o Job acabar com sucesso se `entrada = válidos + quarentena + duplicados` para o lote. Se a conta não fechar, ou se qualquer passo falhar, o Job segue para **Log erro** e **Abort**.

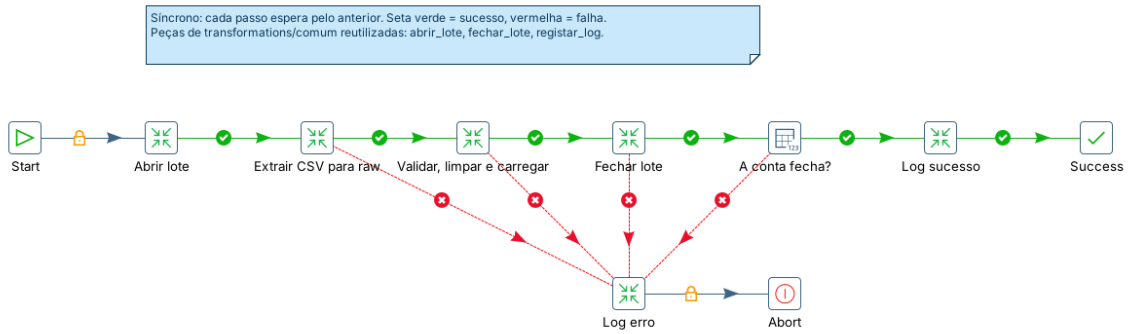


Figura 15 – Job c1_clientes

Fonte: Elaboração própria (captura do Pentaho Spoon).

5.2 `c2_faturas`

`Start` → **Abrir lote** → para cada ficheiro SAF-T: **Validar XSD** (*XSD Validator*, com `src/xsd/saft_simplificado.xsd`).

- Se o ficheiro for válido, segue para **Ler SAF-T** (`c2_01`, com o ficheiro como parâmetro).
- Se não for válido, o ficheiro é **copiado** para `data/quarantine` (*Copy files*) e é inserida uma linha `XSD_INVALIDO` em `quarentena.registo` (entrada *SQL*). O Job continua com o ficheiro seguinte: um ficheiro mau não impede o tratamento dos outros.

Depois: **Exportar JSON e XML** (`c2_02`) → **Fechar lote** → **Gerar resumo XML** (`c2_03`) → **Relatório HTML** (*XSLT*, com `src/xsl/relatorio_auditoria.xsl`) → **Compactar** (*Zip file*: junta o JSON, o XML e o HTML do lote em `pacote_auditoria_loteN.zip`) → **Enviar Email?** (*Simple evaluation* da variável `ENVIAR_EMAIL`).

- Com `S`, a entrada *Mail* envia o relatório com o ZIP em anexo.
- Com `N`, o Job salta o envio.

Em ambos os casos termina com **Log sucesso** → `Success`.

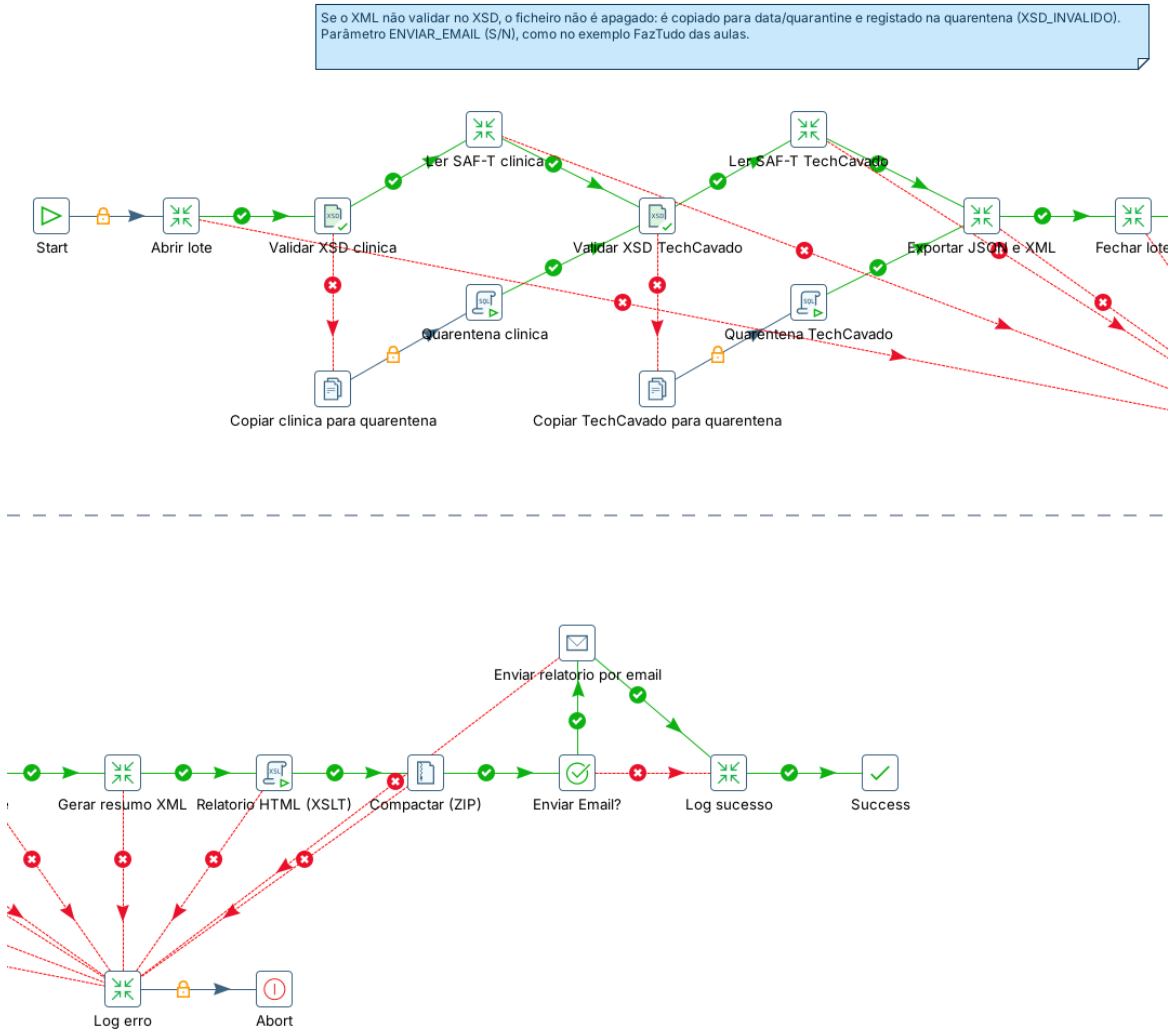


Figura 16 – Job c2_faturas
Fonte: Elaboração própria (captura do Pentaho Spoon).

5.3 `c3_receber_tickets`

`Start` → **Log início** → **Receber tickets (MQTT)** (`c3_01`, que chama `c3_01b` para cada conjunto de mensagens) → `Success`.

É o circuito assíncrono: o Job não sabe quantas mensagens vão chegar, nem quando. Corre em paralelo com o publicador do Node-RED, que envia um ticket a cada 5 segundos, e para-se no botão *Stop* do Spoon.

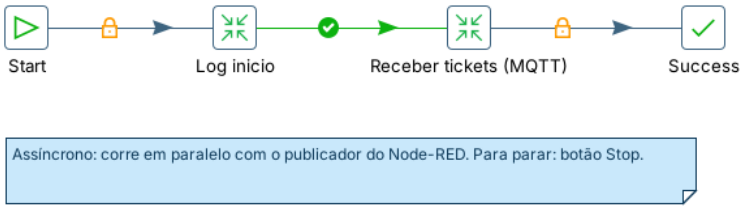


Figura 17 – Job c3_receber_tickets
Fonte: Elaboração própria (captura do Pentaho Spoon).

5.4 `c3_auditoria`

1. **Abrir auditorias** (entrada *SQL*): cria uma execução em `auditoria.execucao` por empresa, com a percentagem `PCT`. A base só aceita valores entre 10 e 20: com outro valor, o Job falha e regista o erro.
2. **Auditar amostra** (`c3_02`).
3. **Contar e criar notificações** (entrada *SQL*): atualiza o total de registos, a amostra, a percentagem observada e as constatações de cada execução, e cria uma notificação `PENDENTE` para cada empresa com constatações (no máximo uma por execução e empresa).
4. **Enviar Email?**
   - Com `S`: **Notificar empresas** (`c3_03`) → **Log notificações enviadas**.
   - Com `N`: **Log notificações pendentes**, com nível `AVISO`. As notificações ficam guardadas e são enviadas na próxima execução com `S`.
5. `Success`.

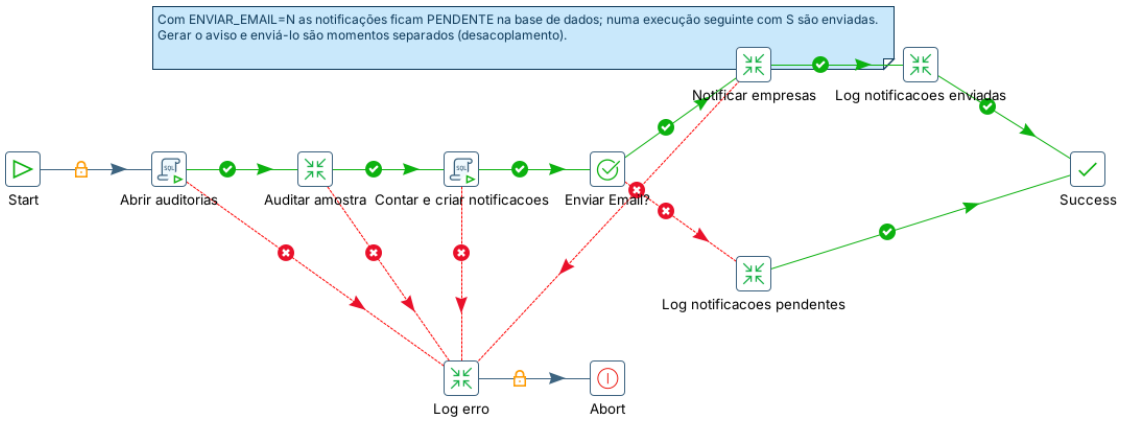


Figura 18 – Job c3_auditoria
Fonte: Elaboração própria (captura do Pentaho Spoon).

5.5 Dashboard e emails

Os alertas que o C3 publica em `piiguard/alertas` são lidos pelo Node-RED e aparecem num dashboard em tempo real: total de alertas, alertas por empresa, distribuição por gravidade e os últimos tickets. O texto é sempre o já mascarado.

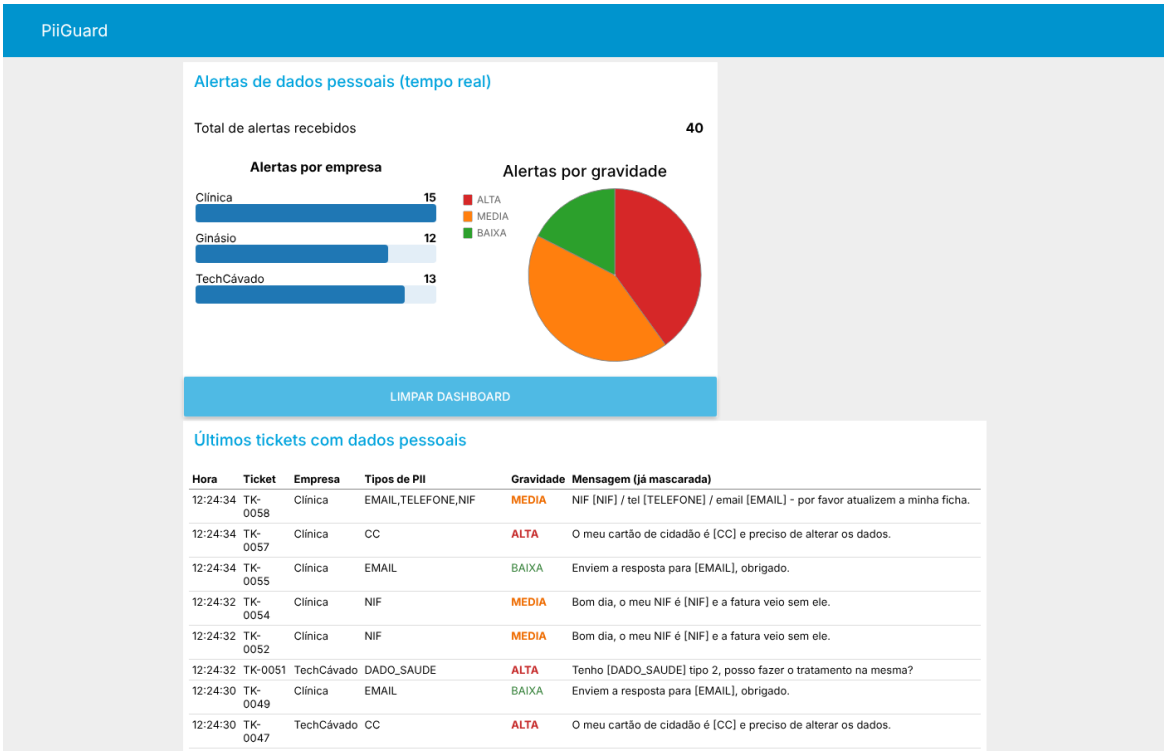


Figura 19 – Dashboard Node-RED com os alertas do C3

Fonte: Elaboração própria.

6. Vídeo de demonstração

O vídeo, com cerca de 6 minutos e sem narração, mostra os três circuitos a correr numa base de dados limpa:

- o Job do C1, com a transformação de validação e o lote reconciliado (110 = 69 + 31 + 10);
- o Job do C2, com o XSD a rejeitar o ficheiro da TechCávado, os ficheiros exportados e o relatório HTML;
- o C3 a receber tickets do Node-RED, com o dashboard a atualizar em tempo real;
- a auditoria por amostragem, as consultas de verificação no pgAdmin e as notificações às empresas, que ficam PENDENTE porque o vídeo foi gravado com ENVIAR_EMAIL=N (o envio é mostrado na apresentação).

Link: https://youtu.be/wKUmVtxbm-M

Figura 20 – QR Code do vídeo de demonstração

Fonte: Elaboração própria.

7. Conclusão e trabalhos futuros

7.1 Resultados obtidos

Os resultados foram comparados com o mapa de estragos (`data/expected/mapa_estragos.csv`), que diz o que o ETL deve fazer com cada erro injetado. A Tabela 8 resume essa conferência.

Tabela 8 – Resultados obtidos e conferência com o mapa de estragos

Circuito	Entrada	Resultado	Conferência
C1 — Clientes	110 registos (2 CSV)	69 válidos em `clean`; 31 em quarentena (12 email, 7 NIF, 6 campo obrigatório vazio, 3 código postal, 2 data, 1 telefone); 10 duplicados	110 = 69 + 31 + 10. Os 31 rejeitados e os motivos coincidem com o mapa
C2 — Faturas	2 ficheiros SAF-T	1 ficheiro rejeitado pelo XSD (copiado para `data/quarantine`); 30 faturas exportadas em JSON e XML com o NIF em hash; 9 faturas de clientes que não existem no CRM	Relatório HTML e ZIP gerados. O envio do ZIP por email depende de ENVIAR_EMAIL; no vídeo foi usado N
C3 — Tickets	63 mensagens MQTT	60 tickets auditados; 1 sem mensagem e 2 repetidos em quarentena; 40 com dados pessoais, todos detetados e mascarados, sem falsos positivos	40 alertas no dashboard
C3 — Auditoria (15 %)	Registos auditáveis: A 91 (39 clientes, 30 faturas, 22 tickets); B 49 (30 clientes, 19 tickets); C 19 (19 tickets)	Amostras de 14, 8 e 3 registos (A, B e C: 15,38 %, 16,33 % e 15,79 %); candidatos não selecionados também guardados; 1 notificação por empresa	Uma notificação por empresa, sem dados pessoais; no vídeo, com ENVIAR_EMAIL=N, ficam PENDENTE

Fonte: Elaboração própria.

Os números da auditoria são os da execução mostrada no vídeo, numa base de dados limpa e já com os tickets do C3: 159 candidatos e 25 registos na amostra. A amostra é aleatória, por isso o número de constatações muda de execução para execução (nessa execução foram 11); o tamanho da amostra, não. A percentagem observada fica um pouco acima da pedida porque o limite é arredondado para cima. Se a auditoria correr antes de chegarem os tickets, os totais são mais baixos (por exemplo, A 70, B 30 e C 0), porque só entra na amostra o que já está na base.

Fizeram-se também testes de falha, resumidos na Tabela 9.

Tabela 9 – Testes de robustez

Teste	Resultado
Correr `01_criar_estrutura.sql` numa base já criada	O script recusa e não apaga nada
`03_migrar_sem_perda.sql` numa base da versão anterior, duas vezes seguidas	Colunas e tabelas acrescentadas; linhas existentes mantidas; a segunda execução não altera nada
Falha forçada a meio da carga do C1 (restrição temporária na quarentena)	Rollback: nada entrou em `clean` nem em `quarentena`; o original ficou em `raw`; lote marcado `FALHOU`
Auditoria com `PCT=25`	Recusada pela base; o Job regista o erro e termina em *Abort*
Envio de email com as credenciais recusadas pelo Gmail	As notificações ficam FALHOU, com a tentativa contada, e são repetidas na execução seguinte (até 3 tentativas), sem duplicados; no C2, o erro fica no log e o Job termina em Abort

Fonte: Elaboração própria.

7.2 Conclusão

O trabalho mostra que o mesmo conjunto de ferramentas serve para três problemas de integração diferentes: juntar fontes com estruturas diferentes (CSV), validar e cruzar um formato normalizado (XML SAF-T, com XSD e XPath) e reagir a eventos em tempo real (MQTT). Nos três, a regra foi a mesma. Nada se apaga, cada registo tem um destino conhecido e a conta de cada lote tem de fechar.

As maiores dificuldades foram:

- as regex no passo *Replace in string*: várias regras sobre o mesmo campo no mesmo passo não funcionam, por isso cada regex ficou num passo próprio, o que, de caminho, torna mais fácil ver o que cada uma faz;
- distinguir uma data com formato errado de uma data impossível (31/02), o que exigiu um passo em JavaScript;
- os duplicados disfarçados, que só aparecem depois de o NIF ser normalizado;
- configurar o envio de email: o passo *Mail* espera nomes de campos e não valores, e as passwords têm de ficar fora dos ficheiros.

7.3 Trabalhos futuros

- Agendar os Jobs (Agendador de Tarefas do Windows com o Kitchen, ou a entrada *Start* com repetição), para a auditoria correr toda a semana.
- Ler os dados diretamente dos sistemas das empresas (API ou base de dados) em vez de ficheiros.

- Melhorar a deteção de dados pessoais em texto livre, incluindo nomes próprios e moradas sem palavras-chave.
- Trocar o hash do NIF por um HMAC com chave secreta, para não ser possível recalculá-lo a partir de uma lista de NIF conhecidos.
- Mostrar o histórico das auditorias no dashboard.

8. Referências bibliográficas

- Casters, M., Bouman, R., & van Dongen, J. (2010). *Pentaho Kettle solutions: Building open source ETL solutions with Pentaho Data Integration*. Wiley.
- Eclipse Foundation. (s.d.). *Eclipse Mosquitto: An open source MQTT broker*. https://mosquitto.org
- Ferreira, L., & Ribeiro, Ó. (2026). *Integração de Sistemas de Informação: Materiais das aulas* (ETL, Tecnologia XML, exemplos aplicados) [Diapositivos]. Universidade Politécnica do Cávado e do Ave.
- Hitachi Vantara. (s.d.). *Pentaho Data Integration 9.4: Documentation*. https://docs.hitachivantara.com
- Kimball, R., & Caserta, J. (2004). *The data warehouse ETL toolkit*. Wiley.
- OpenJS Foundation. (s.d.). *Node-RED documentation*. https://nodered.org/docs/
- Portaria n.º 302/2016, de 2 de dezembro. (2016). *Diário da República*, 1.ª série. Estrutura de dados do ficheiro SAF-T (PT).
- PostgreSQL Global Development Group. (s.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/
- Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho, de 27 de abril de 2016. (2016). *Jornal Oficial da União Europeia*, L 119, 1–88.
- World Wide Web Consortium. (s.d.). *XML Schema 1.0, XML Path Language (XPath) 1.0 e XSL Transformations (XSLT) 1.0*. https://www.w3.org

Anexo A – Declaração de uso de Inteligência Artificial

Neste trabalho usei os assistentes **Claude (Anthropic)** e **Codex (OpenAI)**, entre 12 e 22 de outubro de 2026, como descreve a Tabela 10.

Tabela 10 – Utilização de Inteligência Artificial no trabalho

Para quê	Como foi usado
Escolha do tema e desenho dos circuitos	Discussão das minhas notas das aulas e do enunciado; ajuda a decidir três circuitos diferentes e não demasiado complexos
Instalação e configuração	Ajuda para PostgreSQL, Mosquitto, Node-RED, Pentaho e `kettle.properties`
Dados de teste	Script Python que gera os dados fictícios estragados e o mapa de estragos
Relatório, README e guião do vídeo	Rascunho do texto, revisto e completado por mim. A redação final foi depois reescrita com o Claude (22 de outubro) para melhorar a clareza e o ritmo do texto, sem mudar factos nem números
Revisão e melhorias (Codex)	Migração da base sem perda de dados.

Fonte: Elaboração própria.

Principais prompts (resumidos):

1. "Lembra-te de tudo o que conversámos e debate comigo estas minhas anotações das aulas [requisitos do professor: pelo menos 3 circuitos diferentes, síncrono e assíncrono, regex, limpeza, não destruir dados, saída além de CSV, auditar 10–20 % e enviar email às empresas]."
2. "Dá-me o esquema completo das fases, o cumprimento do enunciado e a avaliação da dificuldade."
3. "Não achas que são muitas plataformas? Não devo usar o Grafana?"
4. "Dá-me o passo a passo da instalação do que me falta."
5. "O professor usa várias transformações e depois chama-as noutra — isto é um Job. Quero também a opção de enviar ou não o email."
6. "O relatório tem de seguir o padrão da UPCA, com o logótipo da instituição."

Todo o conteúdo gerado foi testado e revisto por mim. Sou responsável pelo trabalho entregue.